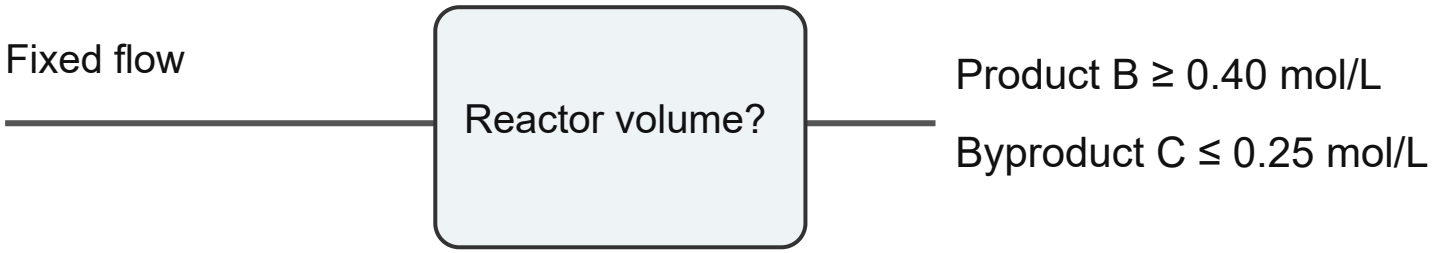
Week 5: From a Predictor to an Optimization Formulation

The smallest reactor that meets product specifications

Product B must reach 0.40 mol/L while byproduct C stays below 0.25 mol/L. At fixed flow, how small can the reactor be?



Changing residence time changes both B and C.

Product specifications and reactor volume minimization

Given a residence time, the network predicts outlet concentrations. Design asks for a residence time that satisfies those concentration requirements.

Longer residence converts A to B, but also converts B to C. Maximizing B and minimizing volume are different objectives.

This week we formulate that choice and express a frozen ReLU computation using linear constraints and binary variables.

Learning objectives for optimization formulation

1. Separate decisions, fixed context, and predicted outputs; write the objective and constraints.
2. Prove the Big-M representation of one ReLU by examining both branches.
3. Derive valid pre-activation bounds from the input domain.
4. Compare a hand-built MILP with the forward pass at identical inputs.

Submit the formulation, verification results, and interpretation of the solution.

Decision variables and fixed context

Role	Value or range in the lab
Decision	Residence time $\tau \in [2, 10]$ min
Fixed context	$T = 330$ K, $C_{Af} = 1.2$ mol/L
Fixed flow	$Q = 1$ L/min
Predicted output	A, B, C concentrations y [mol/L]
Objective	Minimize volume $V = Q\tau$ [L]

Here τ is a design variable selecting volume at fixed flow. Choosing temperature as well requires a model with temperature as an input and bounds over that domain.

An optimization problem in symbols

$$\min_{u,y} J(u, y; d)$$
$$y = f_\theta(u; d),\ g(u, y; d) \leq 0$$

u denotes decisions, d fixed context, and y model outputs. The fitted weights θ are also fixed.

A feasible point satisfies both the network equations and the process constraints. We seek the feasible point with the smallest objective.

CSTR design optimization problem

$$\min_{\tau, y} V = Q\tau$$
$$2 \leq \tau \leq 10, y = f_\theta(\tau)$$
$$y_B \geq .40, y_C \leq .25, y \geq 0$$

With Q = 1 L/min, minimizing τ also minimizes V. Outputs are in mol/L and must be converted to those units before checking specifications.

B ≥ 0.40 is a concentration specification. Purity or production-rate requirements need constraints written from their own definitions.

Reference concentrations for the lab

Assume perfect mixing, constant volume, equal inlet and outlet flow, and consecutive 1:1 reactions A → B → C. At 330 K, $k_1 = 0.2\ \text{min}^{-1}$ and $k_2 = 0.1\ \text{min}^{-1}$.

$$C_A = \frac{1.2}{1 + .2\tau}$$

$$C_B = \frac{.2\tau C_A}{1 + .1\tau}, C_C = .1\tau C_B$$

We evaluate these equations at five residence times to construct a small ReLU model.

The frozen computational graph

$$h^0 = x$$
$$a^\ell = W^\ell h^{\ell-1} + b^\ell$$
$$h^\ell = \max\left(0, a^\ell\right)$$
$$y = W_{\text{out}} h^K + b_{\text{out}}$$

ReLU acts componentwise in each hidden layer. Inputs, intermediate values, and outputs are optimization variables; W and b are numeric constants.

Making weights variable introduces products of W and h, so the MILP derived here would no longer apply.

Equality representation of input scaling

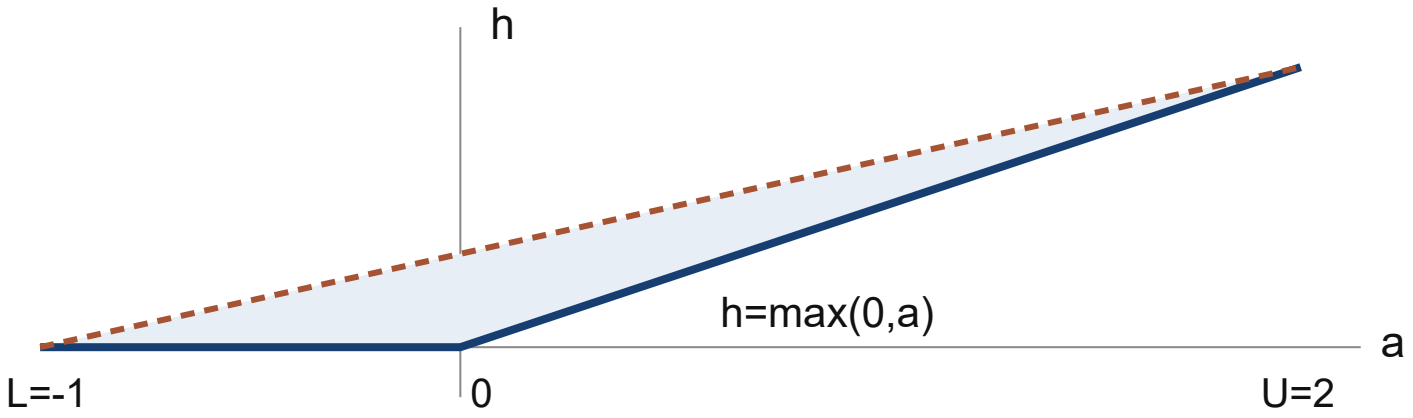
$$x = \frac{\tau - 2}{8}, 0 \le x \le 1$$
$$\tau = 2 + 8x$$

The lab maps τ to dimensionless input x. Optimization uses exactly the offset and scale used to construct the predictor.

If outputs were standardized as well, include y = μ + Ds. This inverse transformation is linear when μ and D are fixed.

Piecewise-linear structure of ReLU

$$h = \max(0, a)$$



The blue line is the exact graph. The shaded region includes points admitted when the binary variable is relaxed.

The ReLU graph and its lower inequalities

$$h \geq 0, h \geq a$$

At a = −0.2, h = 0.4 satisfies both inequalities, although ReLU returns 0.

The objective or a later layer may favor larger h. For an arbitrary network, these lower inequalities alone are insufficient; exact graph modeling also needs branch-specific upper inequalities.

Big-M formulation of one ReLU

Assume valid finite bounds $L \le a \le U$ with $L < 0 < U$.

$$h \ge 0, h \ge a$$
$$h \le a - L\,(1 - z)$$
$$h \le Uz, z \in \{0, 1\}$$

$z = 0$ selects the inactive branch; $z = 1$ selects the active branch. L and U bound this neuron's pre-activation.

Equivalence of the inactive branch (z = 0)

$$h \leq U0 = 0, h \geq 0$$
$$h = 0, a \leq h = 0$$

The remaining upper inequality, 0 ≤ a − L, holds because a ≥ L. This branch therefore represents exactly L ≤ a ≤ 0 and h = 0.

Equivalence of the active branch (z = 1)

$$h \le a - L0 = a, h \ge a$$
$$h = a \ge 0$$

The remaining upper inequality h ≤ U is equivalent to a ≤ U. Thus 0 ≤ a ≤ U and h = a.

At a = 0, z can be either 0 or 1, and the output h is fixed at 0.

Linear representations of stable active and inactive neurons

Valid range	Replacement	Binary variable
$U \leq 0$	$h = 0$	None
$L \geq 0$	$h = a$	None
$L < 0 < U$	Four Big-M inequalities	One

Allowing inputs outside the bounded domain invalidates this simplification. Recheck bounds and classifications when changing the domain.

Interval bounds accounting for weight signs

$$a = \sum_i w_i h_i + b$$
$$L = b + \sum_i \min\left(w_i l_i, w_i u_i\right)$$
$$U = b + \sum_i \max\left(w_i l_i, w_i u_i\right)$$

Assume $l_i \le h_i \le u_i$ for every input. Negative weights reverse which input endpoint gives the minimum or maximum.

Interval arithmetic gives valid bounds, but ignoring correlations between inputs can make them wider than the attainable range.

A bound calculation by hand

$$a = 2x_1 - 3x_2 + .5$$
$$0 \le x_1 \le 1, -1 \le x_2 \le 2$$
$$L = -5.5, U = 5.5$$

The minimum occurs at $(x_1, x_2) = (0, 2)$, and the maximum at $(1, -1)$. Do this calculation before choosing an arbitrary large constant.

Bound validity and formulation correctness

Using $U = 0.5$ at a point whose true pre-activation is $a = 0.7$ excludes $h = 0.7$: even $z = 1$ imposes $h \leq 0.5$.

Wider valid bounds preserve the graph at integer solutions, but can weaken the continuous relaxation.

Observed extrema in the training samples do not automatically bound the entire input domain.

Frozen coefficients of the ReLU interpolant

Linearly interpolate reference concentrations at τ = 2, 4, 6, 8, 10 min. Fix the hidden hinges at those breakpoints and compute output weights from interpolation slopes.

$$a_j = x - t_j, h_j = \max\left(0, a_j\right)$$

$$t = \left[\begin{array}{cccc} 0 & .25 & .50 & .75 \end{array}\right]$$

$$y_k = b_k + \sum_{j=0}^{3} W_{\mathrm{jk}} h_j$$

j = 0, 1, 2, 3 and output k is A, B, or C. This lab constructs the model by interpolation rather than gradient training.

Neuron bounds in the lab model

Neuron	a_j	L_j	U_j	Treatment
0	x	0	1	$h_0 = x$
1	$x - 0.25$	-0.25	0.75	Binary
2	$x - 0.50$	-0.50	0.50	Binary
3	$x - 0.75$	-0.75	0.25	Binary

There are three binary variables. Substituting the affine pre-activations leaves eight continuous variables: one x, four h values, and three outputs.

Matrix representation of Big-M constraints

For neuron 1, $a_1 = x - 0.25$, $L_1 = -0.25$, and $U_1 = 0.75$.

$$h_1 - x \geq -.25$$
$$h_1 - x + .25z_1 \leq 0$$
$$h_1 - .75z_1 \leq 0$$

Set $h_1 \geq 0$ and $0 \leq z_1 \leq 1$ as variable bounds, and declare z_1 integer. The lab assembles the full matrix in the same way.

Conditions for MILP representability

A graph of fixed affine layers, ReLUs, and affine scaling can be expressed by an MILP. The objective and all remaining constraints must also be linear for the entire problem to be an MILP.

Products Qy with both flow and concentration variable, nonlinear kinetics, or sigmoid/tanh operations fall outside this formulation.

The Week 4 projection can also be appended as affine equalities when its coefficient matrices are fixed and its input dependence is affine.

MILP and forward-pass consistency at fixed inputs

Fix one residence time, remove product specifications, and solve the network equations. Compare every h and y with the forward pass.

We checked 41 inputs covering domain endpoints, all breakpoints, both sides of interior breakpoints, and random inputs. The maximum difference was 1.25e-08.

This tests the formulation implementation. Process prediction error requires comparison with separate reference values.

CSTR design optimization results

Quantity	This run
Minimum τ [min]	5.353846
Minimum V [L]	5.353846
Predicted [A, B, C] [mol/L]	[0.584615, 0.400000, 0.215385]
Binary [z_1, z_2, z_3]	[1, 0, 0]
Termination	optimal, status = 0

The B lower bound is active, while the C upper bound has slack. Independent segmentwise linear calculations give the same residence time.

Formulation equivalence and process prediction error

The MILP exactly represents the frozen interpolant. This property does not remove its process approximation error.

At the selected $\tau = 5.353846$ min, the reference concentrations are [0.579495, 0.404137, 0.216369] mol/L, which differ from the predictor.

The solution is optimal for the specified surrogate problem. A process design decision also requires checking reference-process specifications and comparing objective values.

LP relaxation and objective lower bounds

Relaxing z to [0, 1] gives a lower bound on the MILP optimum for minimization. Intermediate values and outputs may nevertheless leave the network graph.

With $L = -1$, $U = 2$, $a = 0$, the relaxed point $z = 1/3$, $h = 2/3$ is feasible, although the ReLU output is 0.

The LP and MILP objective values coincide in this CSTR example. Equality of objectives in one instance does not establish general graph equivalence.

Run the lab and prepare the submission

```
python -m pip install -r requirements.txt
python week05_lab.py --output-dir my-results
```

1. Check one neuron’s bounds and matrix signs by hand.
2. Inspect forward error and termination status in `results.json`.
3. Translate one row of `milp_matrix.csv` back into an equation.
4. Change the B lower bound from 0.40 to 0.41 and explain whether feasibility remains.

The reading companion contains derivations, the weight table, and exercise checks. Download and run the lab bundle.

Sources and assigned reading

Boyd & Vandenberghe, Convex Optimization, §4.1: variables, objectives, and feasible sets.

Anderson et al., Strong mixed-integer programming formulations for trained neural networks, §2.1: the ReLU Big-M formulation.

SciPy milp documentation: bounds, integrality, LinearConstraint, and return status.